

AI in Cybersecurity

A hands-on, two-day instructor-led course for **SOC analysts, detection engineers, and security practitioners** who want to combine cloud security operations with AI-assisted investigation workflows. Over two days, you build a complete security detection and investigation pipeline on AWS — starting from a bare EC2 instance and ending with automated alarms, dashboards, and AI-driven triage that fire without manual intervention.

This is not a lecture-only course. You work as both **attacker and defender**: you simulate realistic threat activity, collect endpoint evidence, enable AWS-native ML detection, stream logs to CloudWatch, and evaluate AI-generated hypotheses like a real SOC analyst.

What You Will Build

By the end of the course, your training AWS account contains a working mini-SOC environment:

Component	Purpose
Amazon Linux 2023 EC2 instance (soc-lab)	Target host with nginx, auditd, and simulated attack artifacts
Endpoint telemetry	auditd (privilege/auth events) + nginx access logs (web reconnaissance)
Amazon Bedrock triage script	Direct AI analysis of raw logs via Python (soc_triage.py)
GuardDuty + Security Hub	AWS ML threat detection and compliance posture aggregation
CloudWatch Logs	Centralized log ingestion via the CloudWatch agent (/soc-lab/audit, /soc-lab/nginx)
CloudWatch Log Analytics	Natural-language Logs Insights query generation
CloudWatch AI Operations	Automated investigations with AI-generated hypotheses
Metric filters + alarms	Signature-based detection on web scans, sudoers changes, and capstone threats
SOC dashboard	Multi-widget soc-lab-web-monitor dashboard for analyst visibility

The arc of the course mirrors production security operations: **generate evidence** → **detect** → **investigate** → **document** → **close**.

Platform and Tooling

Category	Technologies
Cloud platform	AWS (us-east-1 region)
Compute	EC2 (Amazon Linux 2023, t2.medium)

Category	Technologies
Endpoint logging	auditd, journald, nginx
AWS security services	GuardDuty, Security Hub, CloudTrail, IAM, S3
Observability	CloudWatch Logs, Logs Insights, Log Analytics, metric filters, alarms, dashboards
AI / ML	Amazon Bedrock (Claude/Mistral), CloudWatch AI Operations, AI coding assistants
Languages	Python 3.x, Bash
Editor / remote access	VS Code with Remote - SSH extension

Prerequisites

Students should arrive with:

- **Basic Python** — reading files, writing simple scripts, running commands from the terminal
- **Foundational security concepts** — authentication, SSH, privilege escalation, alerts, and incident triage
- **An AWS account** with permissions to launch EC2 instances, create IAM roles, and enable GuardDuty and Security Hub
- **VS Code** with the **Remote - SSH** extension installed

No prior AWS experience is required. Cloud concepts (VPC, security groups, IAM roles) are introduced progressively in Lab 1.1.

Course Structure

Labs are completed **in order**. Each lab builds on artifacts from the previous one. Later labs on each day are supplemental — if time runs short, the class finishes any started lab before moving on.

Day 1 — Environment Setup, Attack Simulation, and AWS-Native Detection

Focus: stand up the lab environment, generate realistic suspicious activity, triage it with AI, and enable AWS-managed threat detection.

Lab	Title	Key Topics
1.1	Instance Setup	EC2 launch, SSH key pairs, security groups, VS Code Remote SSH, Linux file permissions, SetUID binaries, nginx web service
1.2	Suspicious Activity Simulation and AI Triage	Failed SSH brute force, web reconnaissance probes, sudoers backdoor persistence, auditd rules, Bedrock SOC triage script
1.3	AWS AI-Powered Threat Detection	GuardDuty ML detection, Security Hub aggregation, CIS/FSBP compliance findings, S3 misconfiguration detect-and-remediate loop

Day 2 — Log Streaming, Automated Detection, and Capstone

Focus: centralize logs in CloudWatch, build detection pipelines, wire alarms to AI investigations, and independently detect a chosen attack technique.

Lab	Title	Key Topics
2.1	CloudWatch Log Streaming and AI Investigation	IAM instance profile, CloudWatch agent, ACLs for audit log access, Log Analytics natural-language queries, AI Operations investigations
2.2	Security Monitoring Dashboard and Automated Detection	Metric filters, SOC dashboard (7+ widgets), CloudWatch alarms, SNS notifications, alarm-triggered AI investigations
2.3	Attack and Detect Capstone	Independent detection pipeline build, attack simulation with <code>testattacker</code> , AI investigation evaluation, structured incident report

Detailed Lab Walkthrough

Lab 1.1 — Instance Setup

Goal: Create the foundation host that all later labs depend on.

You launch an EC2 instance in `us-east-1` using AWS CloudShell and the AWS CLI. You configure a security group for SSH (port 22) and HTTP (port 80), connect via VS Code Remote SSH with a `.pem` key pair, and practice essential Linux skills: navigation, file permissions, and script execution.

Security concepts covered include:

- **SetUID binaries** — how attackers hunt for privilege escalation paths (`find / -perm -4000`)
- **Key-only SSH authentication** — controlled remote access to cloud hosts
- **nginx web service** — generates HTTP telemetry used in later detection labs

ATT&CK focus: T1021 (Remote Services), T1078 (Valid Accounts)

Checkpoint: EC2 running, VS Code SSH connected, nginx serving the lab page at `http://YOUR_PUBLIC_IP`.

Lab 1.2 — Suspicious Activity Simulation and AI Triage

Goal: Generate controlled attack telemetry and triage it with AI before any CloudWatch setup exists.

You simulate a multi-vector attack from CloudShell:

1. **Failed SSH attempts** — brute-force signal against your EC2 public IP
2. **Suspicious web requests** — curl probes for `/admin`, `/.env`, `/wp-admin`, etc. (404 fingerprint)
3. **Privilege escalation persistence** — drop a NOPASSWD sudoers backdoor file (`/etc/sudoers.d/lab-backdoor`)

You configure `auditd` to watch `/etc/sudoers.d/` for file creation events, collect evidence with `ausearch` and `journalctl`, then write `soc_triage.py` — a Python script that sends the last 50 lines of audit and nginx logs

to **Amazon Bedrock** and receives a structured SOC analysis with MITRE ATT&CK mappings and recommended next steps.

This lab establishes the raw log evidence that Day 2 streams to CloudWatch.

ATT&CK focus: T1110 (Brute Force), T1548 (Abuse Elevation Control Mechanism), T1190 (Exploit Public-Facing Application)

Checkpoint: auditd rule active, backdoor file created, Bedrock triage script prints AI analysis.

Lab 1.3 — AWS AI-Powered Threat Detection

Goal: Enable AWS-managed ML detection that surfaces your Day 1 activity automatically.

You enable **GuardDuty** (ML-based threat detection on CloudTrail, VPC flow logs, DNS) and **Security Hub** (centralized findings mapped to MITRE ATT&CK, CIS Benchmark, and AWS Foundational Security Best Practices).

Expected findings from earlier labs include:

- **UnauthorizedAccess:EC2/SSHBruteForce** — from Lab 1.2 SSH attempts
- Posture findings — open security groups, IAM users without MFA, etc.

You also run a full **misconfiguration** → **detect** → **remediate** loop: create an S3 bucket with public access block disabled, wait for Security Hub control **S3.1** to flag it, then re-enable the block and verify the finding resolves.

Timing note: GuardDuty findings typically appear 10–30 minutes after activity. Security Hub posture findings may take 30–60 minutes on a freshly enabled account.

ATT&CK focus: T1110, T1548

Checkpoint: Both services enabled, at least one finding visible (or noted as pending), S3 misconfiguration remediated.

Lab 2.1 — CloudWatch Log Streaming and AI Investigation

Goal: Move from ad-hoc Bedrock scripts to a persistent, cloud-native investigation pipeline.

You attach an **IAM instance profile** to EC2 (**soc-lab-ec2-monitoring**) granting CloudWatch and SSM permissions, install the **CloudWatch agent**, and configure ACLs so the **cwagent** user can read **/var/log/audit/audit.log**. Logs stream to:

- **/soc-lab/audit** — auditd events
- **/soc-lab/nginx** — web access logs

You then use **CloudWatch Log Analytics** ("Ask AI to write a query") to generate Logs Insights queries in plain English — for example, "Need to find failed SSH attempts" or "Show me logs of privilege escalation attempts."

Finally, you enable **CloudWatch AI Operations**, create investigations from log queries, evaluate AI-generated hypotheses (ruling out false positives is as valuable as escalating real threats), document analyst decisions,

and close investigations with reasons.

ATT&CK focus: T1110, T1548, T1190

Checkpoint: Both log groups ingesting, AI Operations enabled, two investigations created and closed with documented reasons.

Lab 2.2 — Security Monitoring Dashboard and Automated Detection

Goal: Build production-style detection that triggers AI triage automatically — no manual query required.

You create two detection pipelines:

Pipeline 1 — Web reconnaissance (nginx)

- Metric filter on `/soc-lab/nginx` counting 404 responses
- Custom metric: `SOCLab/WebScanAttempts`
- Dashboard `soc-lab-web-monitor` with 7+ widgets (scan count, top paths, source IPs, timelines, AI-generated exploration widget)
- Alarm `web-scan-alarm` (≥ 3 events in 1 minute) → auto-triggers AI Operations investigation

Pipeline 2 — Privilege persistence (auditd)

- Metric filter on `/soc-lab/audit` matching `nametype=CREATE sudoers`
- Custom metric: `SOCLab/SudoersModified`
- Alarm `sudoers-alarm` (≥ 1 event) → auto-triggers AI Operations investigation

You re-run attack simulations to confirm alarms fire and investigations open without manual intervention.

ATT&CK focus: T1190, T1595.002 (Active Scanning), T1548

Checkpoint: Both metric filters and alarms exist, dashboard populated, both alarms triggered AI investigations automatically.

Lab 2.3 — Attack and Detect Capstone

Goal: Synthesize everything — you own the full detect-investigate-document loop independently.

You create a low-privilege `testattacker` account, choose one threat from a menu, and build its complete detection pipeline without step-by-step guidance:

Threat	Attack simulation	Detection pattern	ATT&CK
A — SSH brute force	15 failed SSH logins	<code>exe="/usr/sbin/sshd"</code> <code>res=failed</code>	T1110
B — New account backdoor	<code>useradd backdoor-user</code>	<code>type=ADD_USER</code>	T1136
C — Sensitive file access	<code>cat /etc/shadow</code> as non-root	<code>name="/etc/shadow"</code>	T1003

Threat	Attack simulation	Detection pattern	ATT&CK
D — su abuse	10 failed su - root attempts	exe="/usr/bin/su" res=failed	T1548
E — Cron persistence	Install cron job under testattacker	crontab	T1053

For your chosen threat, you build:

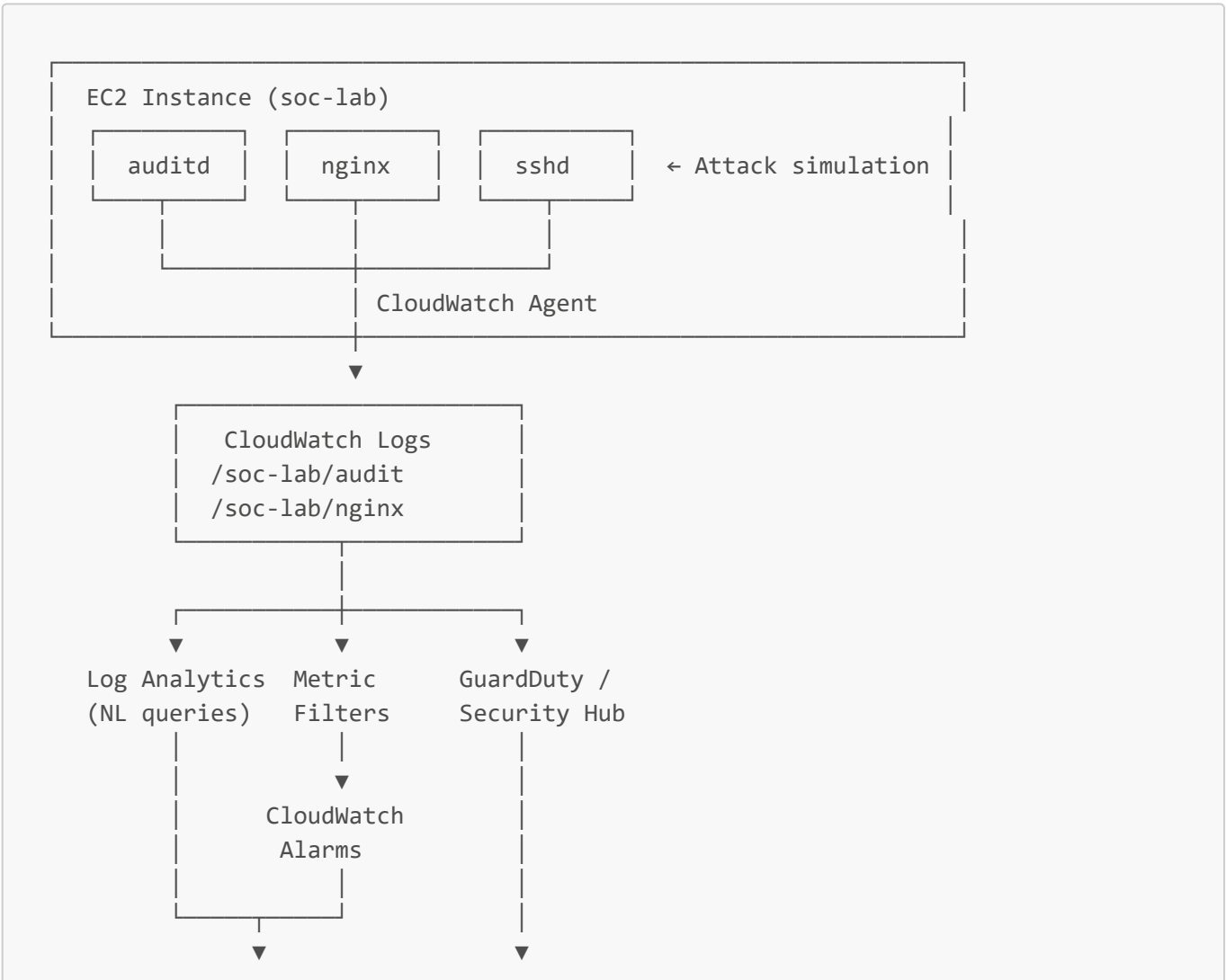
- 1. A **metric filter** on /soc-lab/audit
- 2. A **CloudWatch alarm** wired to AI Operations
- 3. A **dashboard widget** on soc-lab-web-monitor

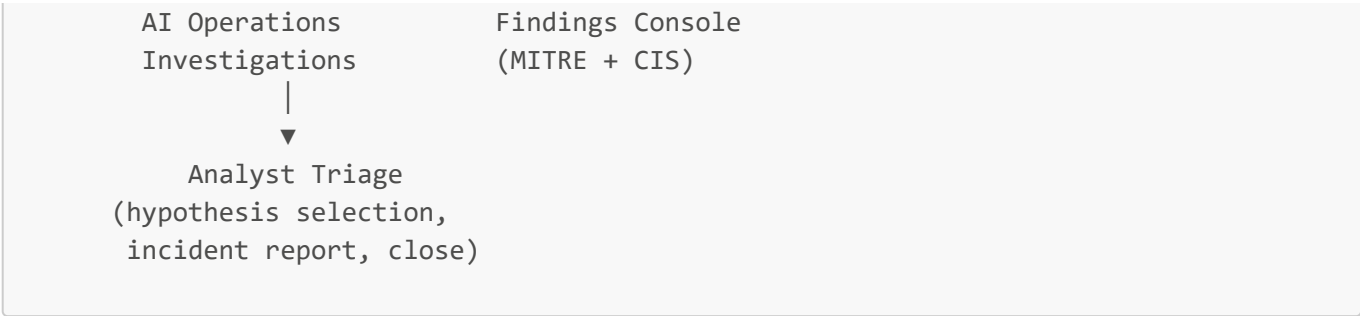
You simulate the attack, verify every pipeline stage fired (log → metric → alarm → investigation), evaluate the AI hypothesis, write a structured **incident report** (timeline, raw log evidence, disposition), and formally close the investigation.

Checkpoint: End-to-end detection confirmed, incident report written, investigation closed.

End-to-End Detection Architecture

The course teaches a simplified but realistic SOC pipeline:





MITRE ATT&CK Coverage

The course maps lab activity to real-world attacker behaviors:

Technique	Where it appears
T1021 — Remote Services	Lab 1.1 SSH access setup
T1078 — Valid Accounts	Lab 1.1 key-based auth; Lab 2.3 testattacker account
T1110 — Brute Force	Lab 1.2 failed SSH; GuardDuty finding; Capstone Threat A
T1190 — Exploit Public-Facing Application	Lab 1.2 web probes; Lab 2.2 nginx detection
T1548 — Abuse Elevation Control Mechanism	Lab 1.2 sudoers backdoor; Lab 2.2 sudoers alarm; Capstone Threat D
T1136 — Create Account	Capstone Threat B
T1003 — OS Credential Dumping	Capstone Threat C (<code>/etc/shadow</code> access)
T1053 — Scheduled Task/Job	Capstone Threat E (cron persistence)
T1595.002 — Active Scanning: Vulnerability Scanning	Lab 2.2 web scan detection

Learning Outcomes

After completing this course, you will be able to:

1. **Launch and harden** a cloud Linux host for security lab work (EC2, security groups, SSH, file permissions)
2. **Simulate controlled attack activity** that produces realistic endpoint telemetry without destructive tools
3. **Configure auditd** to detect privilege escalation and persistence techniques
4. **Triage security logs with AI** using Amazon Bedrock and CloudWatch AI Operations
5. **Enable and interpret** AWS-native ML detection (GuardDuty, Security Hub)
6. **Stream endpoint logs** to CloudWatch using IAM roles and the CloudWatch agent
7. **Write Logs Insights queries** manually and via natural-language AI assistance
8. **Build detection pipelines** — metric filters, alarms, dashboards — that trigger automated AI investigations

9. **Evaluate AI hypotheses** critically, distinguishing real threats from misconfigurations and expected lab activity
 10. **Document and close incidents** with structured reports and analyst disposition
-

Safety Rails

This course uses **controlled, non-destructive simulations** in dedicated training AWS accounts only.

- Run all labs only in your **training AWS account** — never against production systems
 - Do not run destructive tools, malware, or exploit frameworks
 - Keep credentials (**.pem** key files, Bedrock API keys) **local** — do not paste them into AI tools or share them
 - GuardDuty and Security Hub offer a **30-day free trial** — no charges during the lab period
 - Attack simulations (sudoers backdoor, cron jobs, failed logins) are **intentionally left in place** as starting state for subsequent labs — clean up after the course completes
-

Repository Layout

```
AI-in-Cybersecurity-main/
├── README.md           ← This file
├── labs/
│   ├── 1.1-Instance-Setup.md
│   ├── 1.2-Suspicious-Activity-Simulation-and-AI-Triage.md
│   ├── 1.3-AWS-AI-Powered-Threat-Detection.md
│   ├── 2.1-CloudWatch-Log-Streaming-and-AI-Investigation.md
│   ├── 2.2-Security-Monitoring-Dashboard-and-Automated-Detection.md
│   ├── 2.3-Attack-and-Detect-Capstone.md
│   └── glossary.md     ← Key terms and AWS service definitions
└── slides/            ← Course slide deck (instructor materials)
```

Reference

- **Glossary** — Definitions for MITRE ATT&CK terms, SOC operations, Linux endpoint security, AWS services, CloudWatch, and AI-assisted security workflows used throughout the course
-

Getting Started

1. Confirm you meet the [Prerequisites](#) above
2. Select the **us-east-1** region in the AWS console (the lab AML is region-specific)
3. Open [Lab 1.1 — Instance Setup](#) and follow the steps in order
4. Complete each lab's **Lab Checkpoint** before proceeding to the next

The course is designed so that by Lab 2.3, you can build, attack, detect, triage, and document — the core workflow of a modern SOC analyst — entirely on your own.